

SUPPORT

Créer un Starter Kit

Node.js + Express + React + MySQL

Version ES Modules (import/export) + Node.js 18+ (--watch)

 **Objectif : Créer un template Git réutilisable avec la syntaxe JavaScript moderne**

1. ES Modules vs CommonJS

Node.js supporte maintenant nativement les ES Modules, la syntaxe standard de JavaScript.

1.1 Comparaison des syntaxes

	CommonJS (ancien)	ES Modules (moderne)
Import défaut	<code>const express = require('express')</code>	<code>import express from 'express'</code>
Import nommé	<code>const { Router } = require('express')</code>	<code>import { Router } from 'express'</code>
Export défaut	<code>module.exports = User</code>	<code>export default User</code>
Export nommé	<code>module.exports = { query, pool }</code>	<code>export { query, pool }</code>
Fichier local	<code>require('./config/db')</code>	<code>import db from './config/db.js'</code>

⚠ IMPORTANT : Avec ES Modules, l'extension .js est OBLIGATOIRE pour les imports locaux !

1.2 Activer les ES Modules

Ajouter cette ligne dans package.json :

```
{
  "name": "mon-projet",
  "type": "module",    // 🚩 Cette ligne active les ES Modules
  ...
}
```

1.3 Node.js --watch (remplace nodemon)

Depuis Node.js 18, le flag --watch est intégré. Plus besoin d'installer nodemon !

Avant (nodemon)	Maintenant (Node 18+)
<code>npm install -D nodemon</code>	Rien à installer !
<code>"dev": "nodemon server.js"</code>	<code>"dev": "node --watch server.js"</code>



node --watch redémarre automatiquement le serveur quand un fichier change !

2. Structure du projet

```
starter-kit/
├── backend/
│   ├── config/
│   │   └── db.js # Pool MySQL (ES Module)
│   ├── controllers/
│   │   └── auth.controller.js # export { register, login, getProfile }
│   ├── middlewares/
│   │   └── auth.middleware.js # export default authMiddleware
│   ├── models/
│   │   └── user.model.js # export default User
│   ├── routes/
│   │   └── auth.routes.js # export default router
│   ├── .env.example
│   ├── package.json # "type": "module"
│   ├── schema.sql
│   └── server.js
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── contexts/
│   │   ├── pages/
│   │   └── services/
│   └── package.json
└── README.md
```

3. Backend - Configuration

3.1 Initialisation

```
# Créer le dossier
mkdir backend && cd backend

# Initialiser npm
npm init -y

# Installer les dépendances (PAS de nodemon !)
npm install express mysql2 dotenv cors bcrypt jsonwebtoken
```

💡 Plus besoin de nodemon en devDependencies ! Node 18+ a --watch intégré

3.2 package.json complet

```
{
  "name": "starter-kit-backend",
  "version": "1.0.0",
  "type": "module",                                // ➡ ACTIVE LES ES MODULES
  "scripts": {
    "start": "node server.js",
    "dev": "node --watch server.js"                // ➡ REMPLACE NODEMON
  },
  "dependencies": {
    "bcrypt": "^5.1.1",
    "cors": "^2.8.5",
    "dotenv": "^16.3.1",
    "express": "^4.18.2",
    "jsonwebtoken": "^9.0.2",
    "mysql2": "^3.6.5"
  },
  "engines": {
    "node": ">=18.0.0"                             // ➡ EXIGE NODE 18+
  }
}
```

3.3 Variables d'environnement (.env.example)

A screenshot of a code editor window with a dark border. The window has standard window controls (minimize, maximize, close) in the top right corner. The code inside is for a .env.example file, showing configuration for a server, MySQL database, and JWT tokens. The text is color-coded: comments are in red, and values are in yellow or red.

```
# Serveur
PORT=5000
NODE_ENV=development

# MySQL
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=
DB_NAME=starter_kit

# JWT
JWT_SECRET=votre_super_secret_jwt_a_changer_absolument
JWT_EXPIRES_IN=7d
```

4. Backend - Connexion MySQL

config/db.js

```
// config/db.js
import mysql from 'mysql2/promise';

// Création du pool de connexions
const pool = mysql.createPool({
  host: process.env.DB_HOST || 'localhost',
  user: process.env.DB_USER || 'root',
  password: process.env.DB_PASSWORD || '',
  database: process.env.DB_NAME || 'starter_kit',
  waitForConnections: true,
  connectionLimit: 10
});

// Fonction utilitaire pour les requêtes
export async function query(sql, params = []) {
  const [results] = await pool.execute(sql, params);
  return results;
}

// Test de connexion
export async function testConnection() {
  try {
    const connection = await pool.getConnection();
    console.log('MySQL connecté');
    connection.release();
    return true;
  } catch (error) {
    console.error('Erreur MySQL:', error.message);
    return false;
  }
}

export default pool;
```

💡 On utilise **export** pour les fonctions nommées et **export default** pour le pool

5. Backend - Modèle User

models/user.model.js

```
// models/user.model.js
import { query } from '../config/db.js'; // ← Extension .js obligatoire !
import bcrypt from 'bcrypt';

const User = {
  // Trouver par email
  async findByEmail(email) {
    const sql = 'SELECT * FROM users WHERE email = ?';
    const results = await query(sql, [email.toLowerCase()]);
    return results[0] || null;
  },

  // Trouver par ID (sans le password)
  async findById(id) {
    const sql = 'SELECT id, email, firstname, lastname, created_at FROM users WHERE id = ?';
    const results = await query(sql, [id]);
    return results[0] || null;
  },

  // Créer un utilisateur
  async create({ email, password, firstname, lastname }) {
    const hashedPassword = await bcrypt.hash(password, 10);
    const sql = `
      INSERT INTO users (email, password, firstname, lastname)
      VALUES (?, ?, ?, ?)
    `;
    const result = await query(sql, [
      email.toLowerCase(),
      hashedPassword,
      firstname,
      lastname
    ]);
    return { id: result.insertId, email, firstname, lastname };
  },

  // Vérifier le mot de passe
  async verifyPassword(plainPassword, hashedPassword) {
    return bcrypt.compare(plainPassword, hashedPassword);
  }
};

export default User;
```

⚠ N'oublie pas l'extension .js dans : import { query } from '../config/db.js'

6. Backend - Middleware JWT

middlewares/auth.middleware.js

```
// middlewares/auth.middleware.js
import jwt from 'jsonwebtoken';
import User from '../models/user.model.js'; // ← .js !

const authMiddleware = async (req, res, next) => {
  try {
    const authHeader = req.headers.authorization;

    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      return res.status(401).json({ error: 'Token manquant' });
    }

    const token = authHeader.split(' ')[1];
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    const user = await User.findById(decoded.id);
    if (!user) {
      return res.status(401).json({ error: 'Utilisateur non trouvé' });
    }

    req.user = user;
    next();
  } catch (error) {
    if (error.name === 'TokenExpiredError') {
      return res.status(401).json({ error: 'Token expiré' });
    }
    return res.status(401).json({ error: 'Token invalide' });
  }
};

export default authMiddleware;
```


7. Backend - Controller Auth

controllers/auth.controller.js

```
// controllers/auth.controller.js
import jwt from 'jsonwebtoken';
import User from '../models/user.model.js';

// Génère un token JWT
const generateToken = (user) => {
  return jwt.sign(
    { id: user.id, email: user.email },
    process.env.JWT_SECRET,
    { expiresIn: process.env.JWT_EXPIRES_IN || '7d' }
  );
};

// POST /api/auth/register
export const register = async (req, res) => {
  try {
    const { email, password, firstname, lastname } = req.body;

    if (!email || !password || !firstname || !lastname) {
      return res.status(400).json({ error: 'Tous les champs sont requis' });
    }

    const existingUser = await User.findByEmail(email);
    if (existingUser) {
      return res.status(409).json({ error: 'Email déjà utilisé' });
    }

    const user = await User.create({ email, password, firstname, lastname });
    const token = generateToken(user);

    res.status(201).json({ message: 'Inscription réussie', user, token });
  } catch (error) {
    res.status(500).json({ error: 'Erreur serveur' });
  }
};

// POST /api/auth/login
export const login = async (req, res) => {
  try {
    const { email, password } = req.body;

    const user = await User.findByEmail(email);
    if (!user || !(await User.verifyPassword(password, user.password))) {
      return res.status(401).json({ error: 'Identifiants incorrects' });
    }

    const token = generateToken(user);
    res.json({
      user: { id: user.id, email: user.email, firstname: user.firstname, lastname: user.lastname },
      token
    });
  } catch (error) {
    res.status(500).json({ error: 'Erreur serveur' });
  }
};

// GET /api/auth/me
export const getProfile = async (req, res) => {
  res.json({ user: req.user });
};
```

8. Backend - Routes

routes/auth.routes.js

```
// routes/auth.routes.js
import { Router } from 'express'; // ← Import nommé
import { register, login, getProfile } from '../controllers/auth.controller.js';
import authMiddleware from '../middlewares/auth.middleware.js';

const router = Router();

// Routes publiques
router.post('/register', register);
router.post('/login', login);

// Routes protégées
router.get('/me', authMiddleware, getProfile);

export default router;
```

💡 On utilise `import { Router } from 'express'` pour importer uniquement ce dont on a besoin

9. Backend - server.js

```
// server.js
import 'dotenv/config'; // ← Syntaxe ES Module pour dotenv
import express from 'express';
import cors from 'cors';
import { testConnection } from './config/db.js';
import authRoutes from './routes/auth.routes.js';

const app = express();
const PORT = process.env.PORT || 5000;

// Connexion BDD
testConnection();

// Middlewares
app.use(cors({ origin: 'http://localhost:5173', credentials: true }));
app.use(express.json());

// Logger (dev)
if (process.env.NODE_ENV !== 'production') {
  app.use((req, res, next) => {
    console.log(`${new Date().toISOString()} | ${req.method} ${req.url}`);
    next();
  });
}

// Routes
app.get('/', (req, res) => {
  res.json({ message: 'Starter Kit API (ES Modules)', status: 'online' });
});
app.use('/api/auth', authRoutes);

// 404
app.use((req, res) => res.status(404).json({ error: 'Route non trouvée' }));

// Démarrage
app.listen(PORT, () => {
  console.log(`Serveur sur http://localhost:${PORT}`);
});
```

💡 **import 'dotenv/config' charge automatiquement le fichier .env (pas besoin d'appeler .config())**

10. Backend - Schema SQL

```
-- schema.sql
CREATE DATABASE IF NOT EXISTS starter_kit
CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

USE starter_kit;

CREATE TABLE IF NOT EXISTS users (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  email VARCHAR(255) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  firstname VARCHAR(100) NOT NULL,
  lastname VARCHAR(100) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

  INDEX idx_email (email)
) ENGINE=InnoDB;

-- TODO: Ajouter vos tables ici (rooms, reservations...)
```

Pour exécuter le script :

```
mysql -u root -p < backend/schema.sql
```

11. Récapitulatif ES Modules

Règle	Exemple
Activer ES Modules	"type": "module" dans package.json
Import par défaut	import express from 'express'
Import nommé	import { Router } from 'express'
Import fichier local	import User from './models/user.model.js'
Extension obligatoire	Toujours .js pour les fichiers locaux !
Export par défaut	export default User
Export nommé	export { query, testConnection }
Charger dotenv	import 'dotenv/config'
Mode watch	node --watch server.js

⚠ ERREUR COURANTE : Oublier l'extension .js dans les imports locaux !

```
// ❌ ERREUR - Va planter
import User from './models/user.model';

// ✅ CORRECT
import User from './models/user.model.js';
```

12. Créer et utiliser le Template

12.1 Publier sur GitHub

```
git init
git add .
git commit -m "Initial commit - Starter Kit ES Modules"
git remote add origin https://github.com/username/starter-kit.git
git push -u origin main
```

12.2 Marquer comme Template

Sur GitHub : Settings > ☒ Template repository

12.3 Utiliser le Template

```
# Cloner le projet
git clone https://github.com/username/starter-kit.git mon-projet
cd mon-projet
rm -rf .git && git init

# Backend
cd backend
cp .env.example .env
npm install
npm run dev          # ← Utilise node --watch

# Frontend (autre terminal)
cd frontend
cp .env.example .env
npm install
npm run dev
```

13. Checklist finale

☐	package.json	"type": "module" ajouté
☐	package.json	Script: "dev": "node --watch server.js"
☐	package.json	"engines": { "node": ">=18.0.0" }
☐	Imports	Tous les .js pour les fichiers locaux
☐	Exports	export default ou export { ... }
☐	dotenv	import 'dotenv/config' en haut de server.js
☐	server.js	npm run dev fonctionne
☐	MySQL	Connexion pool testée
☐	Auth	Register, Login, GetProfile OK

Formation DWWM - La Plateforme_